

Mortgage Engine Implementation Walkthrough

How the Go port sizes a loan: the field-presence `calc` dispatch, the annuity math, the secant APR solver, and the two-variable Newton crossover comparison.

Scope. The Go mortgage package `internal/finance/mortgage/` (`mortgage.go`, `rowgen.go`, `advisories.go`), the API wiring in `internal/api/handlers.go`, and the shared math in `internal/finance/interest/`.

Audience. Engineers maintaining or extending the Go port. Every claim is anchored to a `file:line` reference in `internal/`.

Lineage. A faithful port of the DOS `Mortgage.pas`; for the original algorithm see `legacy/code_docs/mortgage_dos_walkthrough.pdf`. This document explains the Go implementation.

Contents

0 How to read this document

1 Entry & data model

1.1 The pointer-field request · 1.2 The MtgLine row · 1.3 The continuous-rate boundary

2 The Calc dispatch

2.1 Summation · 2.2 The down-payment triad · 2.3 Price ↔ Monthly & balloon

3 Early termination & APR

4 APR crossover comparison

5 Row generation (what-if)

6 Math primitives & the monthly assumption

7 Appendix: call graph & file:line index

0 How to read this document

The Go mortgage engine sizes a loan behind a REST API. As with every Per%Sense screen, the caller leaves one field blank and the engine solves for it: leave the monthly payment blank to size a payment; leave the price blank to find the most expensive house a payment can buy; leave a balloon amount blank to size a balloon. Two comparison features sit on top: a single-loan **APR** (true yield once points are folded in) and a two-loan **APR crossover**.

Citation convention

References look like `mortgage.go:184` and resolve under `internal/finance/mortgage/` unless another path is named (e.g. `api/handlers.go:478`). Pascal lineage is noted as `(Mortgage.pas:NNN)`.

Two things to internalize. (1) Summation `mortgage.go:115` is the annuity factor at the heart of every formula. (2) The stored Rate is a *continuously-compounded* rate; the API converts to/from the displayed nominal-monthly yield at the boundary. The engine is monthly-only — the /12 is hard-coded throughout.

1 Entry & data model

1.1 The pointer-field request

The calc entry is `HandleMortgageCalc` [api/handlers.go:478](#), decoding a `MortgageRequest` [api/handlers.go:55–71](#) whose optional inputs are **pointers** so an omitted key differs from a supplied zero. The handler maps each non-nil pointer to an `MtgLine` field with status `InOutInput`; a nil pointer leaves the status at `StatusEmpty` [api/handlers.go:491–538](#). That presence/absence is the field-presence signal `Calc` reads.

1.2 The `MtgLine` row

The calculation row is `MtgLine` [mortgage.go:44–68](#) (float64 fields, distinct from the decimal storage type). Each value carries an `int8` status:

Field	Meaning
Price / Points	purchase price / origination points (fraction)
Pct / Cash / Financed	the mutually-exclusive down-payment triad: down fraction / cash out of pocket / loan principal
Years / Rate	term / the continuous true rate (see §1.3)
Tax / Monthly	flat monthly escrow (not amortized) / total monthly payment
When / HowMuch / BalloonStat	balloon timing in years / amount / status enum (Known/Unk/Blank)

Note the mortgage row stores balloon timing as `When int` (years), not a date — the mortgage `Calc` path uses no `dateutil` date arithmetic.

1.3 The continuous-rate boundary

`MtgLine.Rate` is a continuous rate; the UI shows a nominal-monthly yield. Conversion happens only at the API boundary: inbound `LoanRateToTrueRate` [api/handlers.go:521](#), outbound `TrueRateToLoanRate` [api/handlers.go:550](#). Both wrap the interest package with a hard-coded `n=12` [mortgage.go:159–174](#): $\text{RateFromYield}(y, 12) = 12 \cdot \ln(1+y/12)$ and $\text{YieldFromRate}(r, 12) = 12 \cdot (e^{r/12} - 1)$. A port that stores the nominal yield in `Rate` corrupts every formula. The APR result is already a loan rate (the solver converts before returning), so the handler does not re-convert it [api/handlers.go:561–566](#).

2 The `Calc` dispatch

`Calc(m)` [mortgage.go:184–309](#) mutates a copy and dispatches purely on `*Status == InOutInput`. Pre-flight: Years must be positive; balloon status is set from field presence (both `When+HowMuch` → `Known`; `When` only → `Unk`, solve it; `HowMuch` without `When` → error); a high rate or `Financed > Price` raises a non-blocking warning [mortgage.go:189–230](#).

2.1 Summation — the annuity factor

`Summation(r, t)` [mortgage.go:115–132](#) is the present value (per unit monthly payment) of $12t$ monthly payments at continuous rate r :

$$\text{Summation} = f \cdot (1 - \text{last}) / (1 - f), \quad f = e^{-r/12}, \quad \text{last} = e^{-r \cdot t} \rightarrow 12t \text{ as } r \rightarrow 0$$

The teeny branch returns the undiscounted count $12t$. The hard-coded `/12` is the monthly-payment assumption baked into the engine.

2.2 The down-payment triad

With `Price` known, `computeCashPctAndFinanced` [mortgage.go:314–342](#) derives the two of `{Pct, Cash, Financed}` that weren't input (status `InOutOutput`):

$$\begin{aligned} \text{Cash} &= \text{Price} \cdot (\text{Pct} + (1 - \text{Pct}) \cdot \text{Points}) \quad \cdot \quad \text{Financed} = \text{Price} \cdot (1 - \text{Pct}) \\ \text{from Cash: Pct} &= (\text{Cash} / \text{Price} - \text{Points}) / (1 - \text{Points}) \quad \cdot \quad \text{from Financed: Pct} = 1 - \text{Financed} / \text{Price} \end{aligned}$$

A `Pct ≥ 0.995` result ($\approx 100\%$ down) is an error.

2.3 Price ↔ Monthly & balloon

With the triad, `Years` and `Rate` known, the main block first values any known balloon (`balloonval = HowMuch · e-Rate·When`), then solves the blank [mortgage.go:241–305](#):

Known	Solve	Formula	Line
Price + triad	Monthly	$(\text{Price}(1 - \text{Pct}) - \text{balloonval}) / S(\text{Rate}, \text{Years}) + \text{Tax}$	mortgage.go:267
Monthly + Pct	Price	$((\text{Monthly} - \text{Tax}) \cdot S + \text{balloonval}) / (1 - \text{Pct})$	mortgage.go:281
Monthly + Cash	Price	$\text{Cash} + (1 - \text{Points}) \cdot ((\text{Monthly} - \text{Tax}) \cdot S + \text{balloonval})$	mortgage.go:281
Price + Monthly, balloon Unk	HowMuch	$\text{bv} = \text{Price}(1 - \text{Pct}) - (\text{Monthly} - \text{Tax}) \cdot S - \text{balloonval}; \text{HowMuch} = \text{bv} \cdot e^{\text{Rate} \cdot \text{When}}$	mortgage.go:346

The flat `Tax` is added after amortization, never financed. Both `Price` and `Monthly` given (no unknown balloon) is over-determined → error. A `balloonCalc` solve deliberately leaves the row `Unk` so a second `CALC` behaves identically.

3 Early termination & APR

Two functions value a loan paid off early — the basis of the APR solver. `TerminalBalloon(ei, t)` [mortgage.go:367–387](#) is the outstanding balance t years out (including the payment due then):

$$TB(t) = [\text{Financed} - (\text{Monthly} - \text{Tax}) \cdot S(\text{Rate}, t - 1/12) - [\text{paid?}] \cdot \text{HowMuch} \cdot e^{-\text{Rate} \cdot \text{When}}] \cdot e^{\text{Rate} \cdot t}$$

`ValueOfPaymentsForTerminatedLoan(ei, r, t)` [mortgage.go:393–421](#) is the PV (at loan date, at an arbitrary trial rate r) of all cash if the loan terminates at t — discounting the terminal payoff at the *trial* rate is what lets the solver search for the yield.

IterateToFindAPR [mortgage.go:428–468](#) is a secant (finite-difference Newton) solve for the rate at which the discounted cash flows equal the net cash advanced:

```
target = Financed · (1 - Points) · first guess apr = Rate + Points/Years
newdelta = (target - value) · delta / (value - oldvalue) → report YieldFromRate(apr, 12)
```

≤ 20 iterations; converged when $|\text{delta}| < \text{tiny}$. `FullTermAPR` [mortgage.go:472](#) uses $t = \text{Years} + 1/12$ (held to maturity); `OneMonthAPR` [mortgage.go:480](#) is the closed-form worst case (payoff at the first payment). The handler calls `FullTermAPR` when `EnoughDataForAPR` holds [api/handlers.go:560](#); non-convergence surfaces as a warning.

4 APR crossover comparison

A **crossover** is the holding period at which two loans have equal effective APR. CompareAPRs

`mortgage.go:505–574` first runs the **dominance test**: a loan that wins at *both* the one-month and full-term horizons is “always better” (no crossover exists) `mortgage.go:531–540`. Otherwise the ordering must flip somewhere between, and the crossover solver finds it.

`iterateToFindCrossoverAPRandTime` `mortgage.go:580–705` solves simultaneously for the rate r and time t at which both loans price to par. The per-loan residual is $1 - V(r, t) / (\text{financed} \cdot (1 - \text{points}))$ `mortgage.go:583`; a 2×2 Jacobian is built by finite differences and a Newton step taken via the inverse Jacobian (Cramer's rule):

$$\det = \partial T_1 / \partial t \cdot \partial T_2 / \partial r - \partial T_1 / \partial r \cdot \partial T_2 / \partial t$$

$$dr = (\partial T_2 / \partial t \cdot T_1 - \partial T_1 / \partial t \cdot T_2) / \det, \quad dt = (-\partial T_2 / \partial r \cdot T_1 + \partial T_1 / \partial r \cdot T_2) / \det$$

First guess $t = 0.25 \cdot (\text{Years1} + \text{Years2})$, rate seeded from the mean APR; ≤ 40 iterations to both residuals < teeny; report `YieldFromRate(r, 12)` with t floored to the previous whole month `mortgage.go:591–704`.

Balloon-date discontinuity fallback. The terminated-loan value is discontinuous at a balloon date, so the Newton iteration can stall. `tryBalloonDates` `mortgage.go:721–748` samples each balloon-bearing loan's APR just before and just after the date; if the APR ordering between the two loans flips across it, the crossover is pinned to that balloon date. A port that omits this fails to converge on balloon loans.

5 Row generation (what-if)

What-if tables vary one input across rows and re-solve the dependent field. EnoughDataForRowGeneration [rowgen.go:38](#) requires the base row to have *computed* a Price, Monthly, or HowMuch (status InOutOutput) — that output field is the one re-solved per row. GenerateRows(base, vary, inc, n) [rowgen.go:58–107](#) bumps the varied field by inc each row (bumpField), blanks the base's output field, and re-runs Calc. bumpField [rowgen.go:111–143](#) applies a rate increment in *loan-rate (yield) space*, not continuous space, so “+0.5%” means 0.5% of nominal rate. GenerateGrid [rowgen.go:156](#) sweeps a second axis. Both axes are bounded by MaxWhatIfRows = 1000.

6 Math primitives & the monthly assumption

The shared interest package: `Exxp/Lnn` [interest/math.go:43 / 67](#) (safe exp/ln with Taylor guards near the origin), and `RealPerYr/YieldFromRate/RateFromYield` [interest/rates.go:42 / 63 / 79](#) ($n(e^{r/n}-1)$ and $n \cdot \ln(1+y/n)$).

The engine is monthly-only. Every mortgage call passes `n=12`: `Summation` uses $f = e^{-r/12}$ and the zero-rate fallback `12t`; the boundary converters and APR reporters all use 12. So `RealPerYr` returns 12 throughout and `YieldFromRate/RateFromYield` reduce to the nominal-monthly↔continuous conversions. The `yrdays` (360 vs 365.25) flows in only via the APR Basis and is otherwise inert.

7 Appendix: call graph & file:line index

POST /api/mortgage/calc → HandleMortgageCalc	api/handlers.go:478
└ build MtgLine (pointer fields → status)	api/handlers.go:491
└ LoanRateToTrueRate (loan rate → continuous)	api/handlers.go:521 / mortgage.go:159
└ mortgage.Calc(m)	mortgage.go:184
└ balloon classification	mortgage.go:208
└ computeCashPctAndFinanced (Pct↔Cash↔Financed)	mortgage.go:314
└ Summation $f \cdot (1 - \text{last}) / (1 - f)$	mortgage.go:115
└ solve Monthly Price balloonCalc	mortgage.go:267 / 281 / 346
└ EnoughDataForAPR → FullTermAPR	api/handlers.go:560 / mortgage.go:472
└ IterateToFindAPR (secant)	mortgage.go:428
└ ValueOfPaymentsForTerminatedLoan → TerminalBalloon	mortgage.go:393 / 367
└ TrueRateToLoanRate (continuous → loan rate)	api/handlers.go:550 / mortgage.go:171
POST /api/mortgage/compare → HandleMortgageCompare	api/handlers.go:625
└ CompareAPRs	mortgage.go:505
└ FullTermAPR ×2, OneMonthAPR ×2 (dominance)	mortgage.go:518 / 480
└ iterateToFindCrossoverAPRAndTime (2-var Newton)	mortgage.go:580
└ tryBalloonDates (discontinuity fallback)	mortgage.go:721
POST /api/mortgage/whatif → GenerateRows / GenerateGrid	rowgen.go:58 / 156

Symbol	Reference
HandleMortgageCalc / boundary converters	api/handlers.go:478 · mortgage.go:159/171
Calc / computeCashPctAndFinanced / balloonCalc	mortgage.go:184 / 314 / 346
Summation	mortgage.go:115
TerminalBalloon / ValueOfPaymentsForTerminatedLoan	mortgage.go:367 / 393
IterateToFindAPR / OneMonthAPR / FullTermAPR	mortgage.go:428 / 480 / 472
CompareAPRs / crossover / tryBalloonDates	mortgage.go:505 / 580 / 721
GenerateRows / bumpField / GenerateGrid	rowgen.go:58 / 111 / 156

Per%Sense Go port. Citations resolve under `internal/`. For the original DOS algorithm, see `legacy/code_docs/mortgage_dos_walkthrough.pdf`. Companion Go walk-throughs: amortization, present value, actuarial.